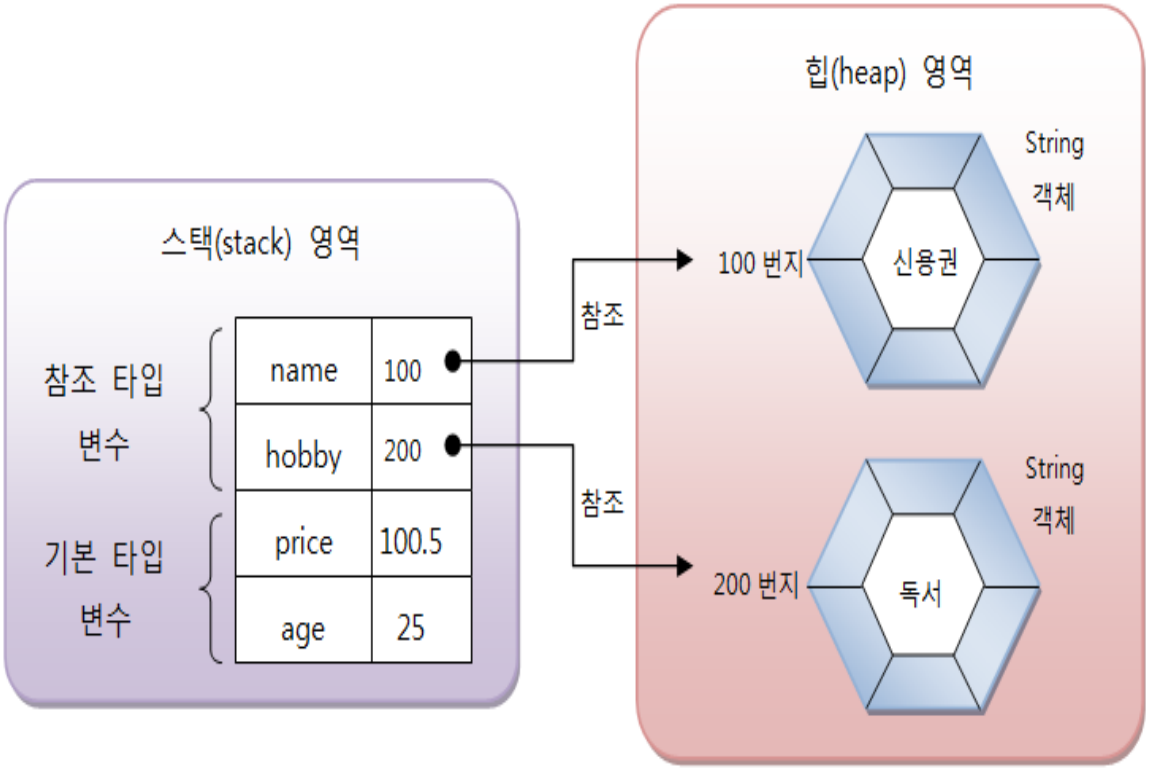
참조타입

데이터 타입

- 변수의 메모리 사용
  - 기본 타입 변수 : 실제 값을 변수 안에 저장
  - 참조 타입 변수 : 주소를 통해 객체를 참조

[기본 타입 변수]
int age = 25;
double price = 100.5;

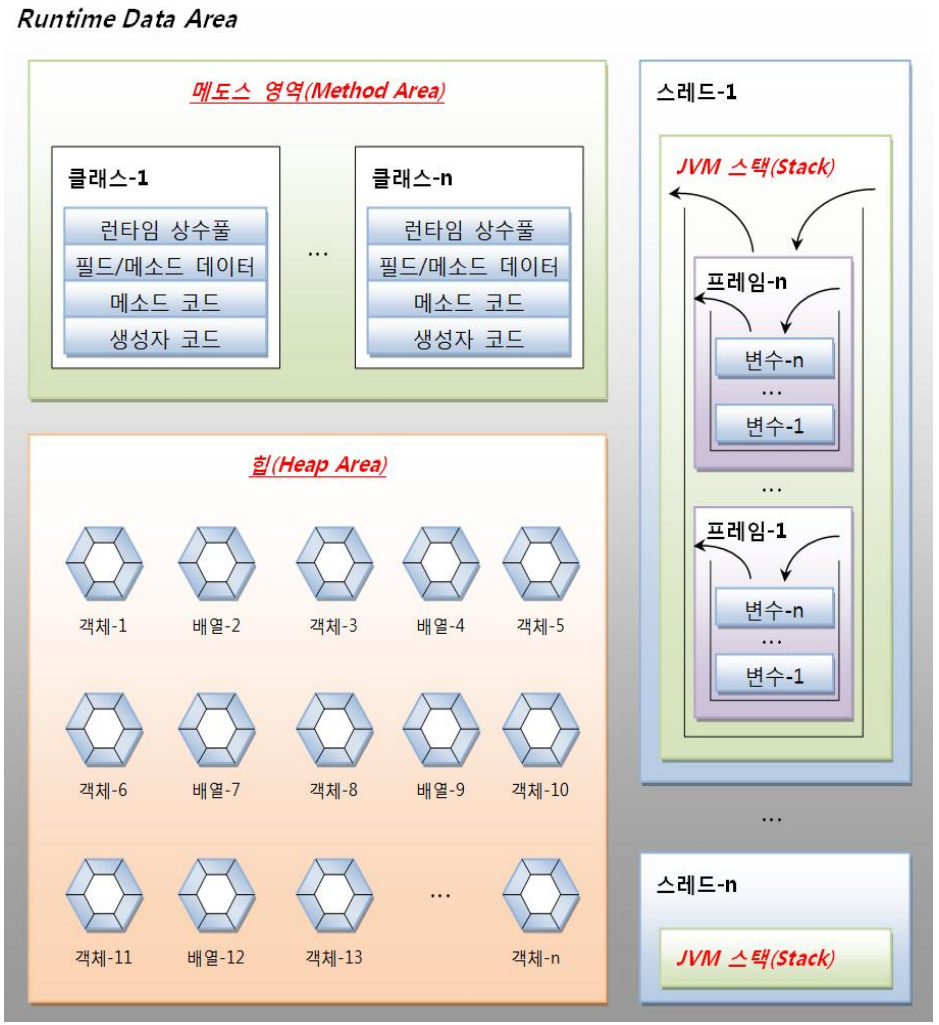
[참조 타입 변수]
String name = "신용권"
String hobby = "독서";



메모리 사용 영역 (1) – JVM 메모리 구조

- JVM이 OS에서 할당 받은 메모리 영역(Runtime Data Area)

  - Method 영역
    - 클래스 로더에 의해 로딩된 바이트 코드 내용 분석후 저장
    - JVM이 시작할 때 생성되고 모든 스레드가 공유
  - Heap 영역
    - 클래스의 객체나 배열 저장
    - 스택 영역에서 참조
    - 아무런 참조하는 대상이 없을 경우 Garbage collector에 의해 회수
  - Stack 영역
    - 각 스레드마다 하나씩 존재
    - 메소드 호출시 프레임(Frame) 추가하고, 종료되면 제거



메모리 사용 영역 (2) – Stack 영역

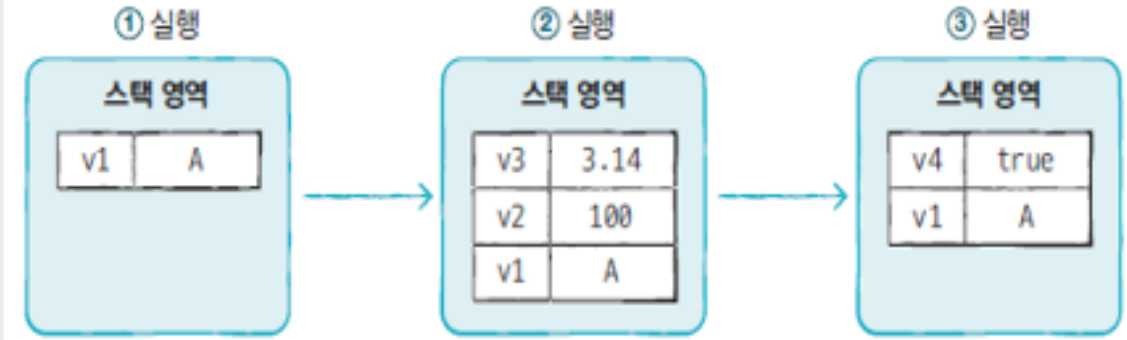
- Stack 영역

  * 기본 타입

```
char v1 = 'A';          ——— ①

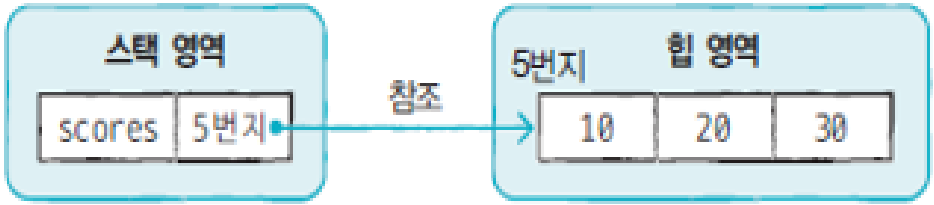
if (v1 == 'A') {
   int v2 = 100;
   double v3 = 3.14;    ——— ②
}

boolean v4 = true;      ——— ③
```



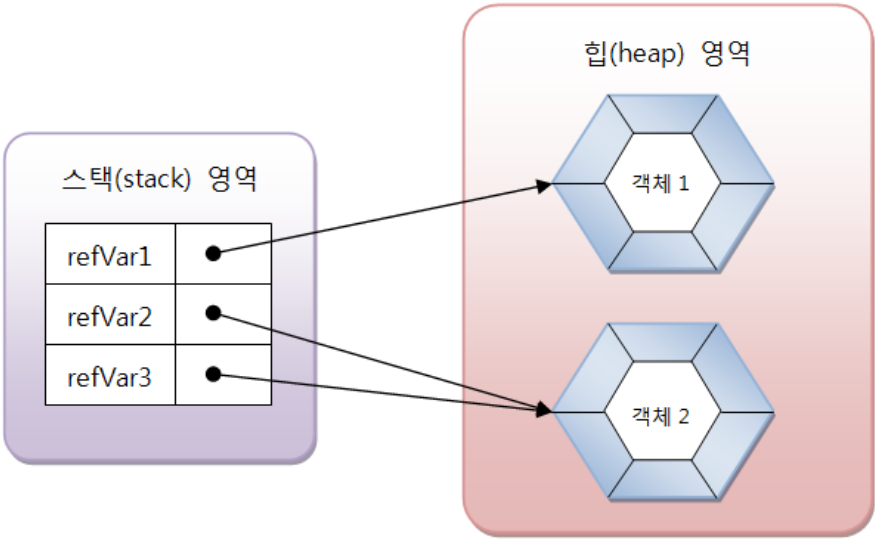
  * 참조 타입

```
int[] scores = {10, 20, 30};
```



참조 변수의 ==, != 연산

- 변수의 값이 같은지 다른지 비교
  - 기본 타입: byte, char, short, int, long, float, double, boolean
    - 의미 : 변수의 값이 같은지 다른지 조사
  - 참조 타입: 배열, 열거, 클래스, 인터페이스
    - 의미 : 동일한 객체를 참조하는지 다른 객체를 참조하는지 조사

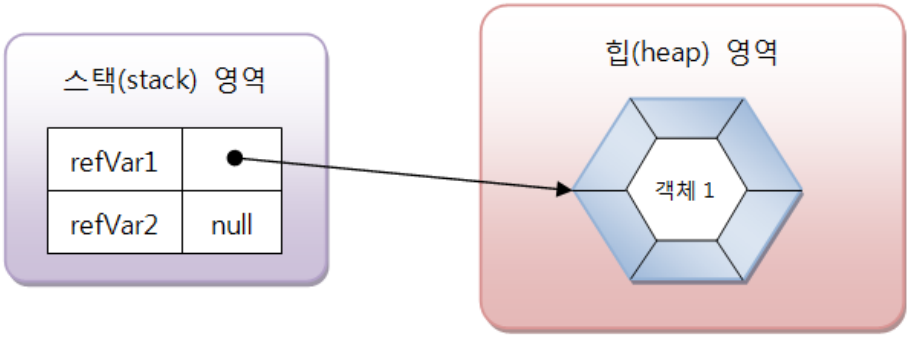


refVar1 == refVar2	결과:	false
refVar1 != refVar2	결과:	true

refVar2 == refVar3	결과:	true
refVar2 != refVar3	결과:	false

null의 의미와 사용

- null(널)
  - 변수가 참조하는 객체가 없을 경우 초기값으로 사용 가능
  - 참조 타입의 변수에만 저장가능
  - null로 초기화된 참조 변수는 스택 영역에 생성



  - ==, != 연산 가능

```
refVar1 == null     결과:  false
refVar1 != null     결과:  true
```

```
refVar2 == null     결과:  true
refVar2 != null     결과:  false
```

NullPointerException

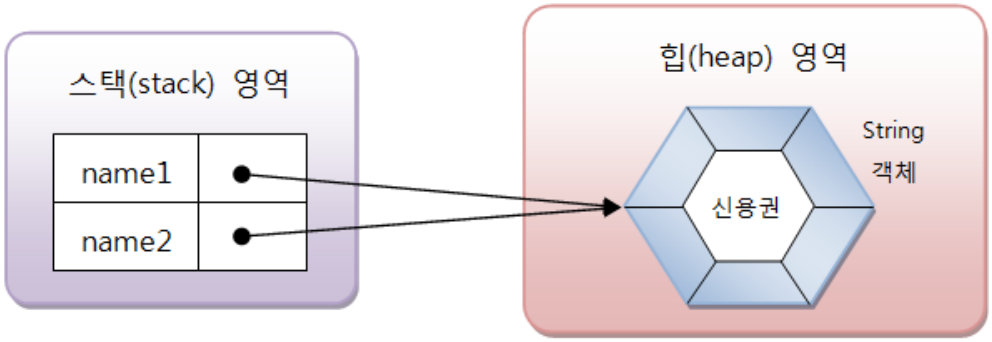
- NullPointerException의 의미
  - 예외(Exception)
    - 사용자의 잘못된 조작이나 잘못된 코딩으로 인해 발생하는 프로그램 오류
  - NullPointerException
    - 참조 변수가 null 값을 가지고 있을 때
      - 객체의 필드나 메소드를 사용하려고 했을 때 발생

```
String str = null;
System.out.println("총 문자수: " + str.length());    //NullPointerException
```

String 타입 (1) – 객체 생성과 비교

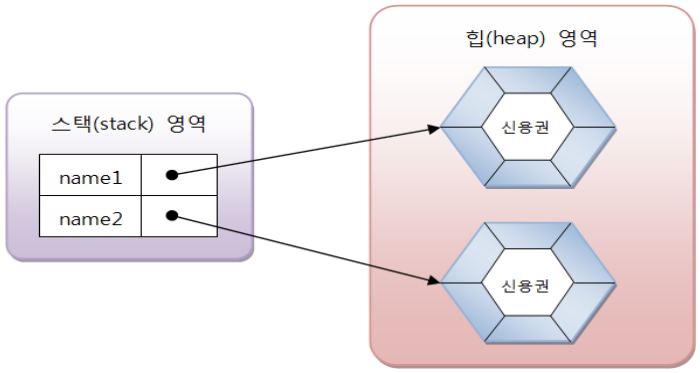
- 문자열 리터럴이 동일하다면 String 객체 공유

```
String name1 = "신용권";
String name2 = "신용권";
```



- new 연산자를 이용한 String 객체 생성
  - 힙 영역에 새로운 String 객체 생성

```
String name1 = new String("신용권");
String name2 = new String("신용권");
```



- String의 equals() 메소드를 이용한 비교

boolean result = name1.equals(name2);

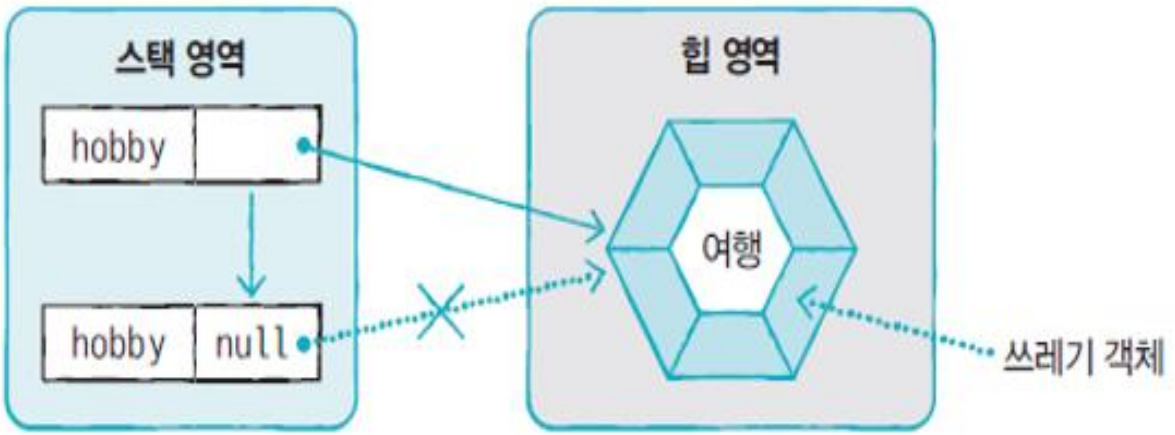
String 타입 (2) – null과 가비지 컬렉션

- String 변수의 초기값으로 null 대입
  - String 변수가 참조하는 객체가 없음을 의미

```
String hobby = null;
```

```
String hobby = "여행";
hobby = null;
```

  - 참조를 잃은 String 객체는 Garbage Collector를 통해 메모리에서 자동 제거됨



배열

자바 배열

- 배열(array)
  - 데이터를 연속된 공간에 나열하고 각 데이터에 인덱스(index)를 부여한 자료구조
    - 배열을 이용하면 한 번에 많은 메모리 공간 선언 가능
    - 원소 데이터들이 순차적으로 저장됨
    - 인덱스를 이용하여 원소 데이터 접근
    - 반복문을 이용하여 처리하기에 적합한 자료 구조
  - 배열 인덱스는 0부터 시작
  - 한번 생성된 배열은 길이를 늘리거나 줄일 수 없음
  - 배열은 생성하면 디폴트 값으로 초기화 됨

배열 선언

- 배열을 사용하기 위해 배열 변수 선언

타입[ ] 변수;	타입 변수[ ];
int[] intArray; double[] doubleArray; String[] strArray;	int intArray[]; double doubleArray[]; String strArray[];

- 배열 변수는 참조 변수이므로 null로 초기화 가능

  int[] intArray=null;

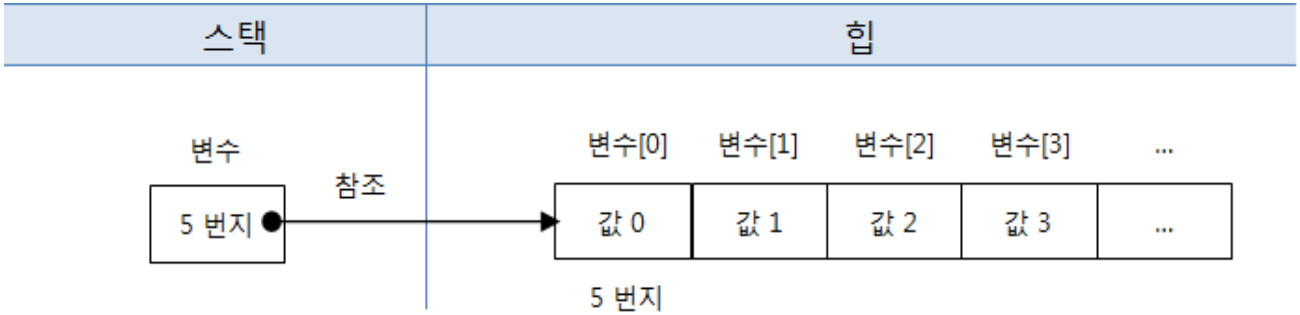
  - 배열 변수가 null 값을 가진 상태에서 항목에 접근 불가
    - 변수[인덱스] (intArray[2]) 사용 못함 ➔ NullPointerException 발생

배열 생성 (1) – 값 리스트로 생성

- 값 리스트로 배열 생성
  - 배열 변수 선언과 동시에 값 리스트 대입

```
데이터타입[] 변수 = { 값 0, 값 1, 값 2, 값 3, ... };
```

```
int[] intArray = {1,2,3,4,5};
```



  - 변수 선언 후 값 리스트 대입

```
데이터타입[] 변수;
변수 = new 타입[] { 값 0, 값 1, 값 2, 값 3, ... };
```

```
int[] intArray=null;
intArray = new int[] {1,2,3,4,5};
```

- 주의 – **오류 발생**

```
int[] intArray=null;
intArray = {1,2,3,4,5};
```

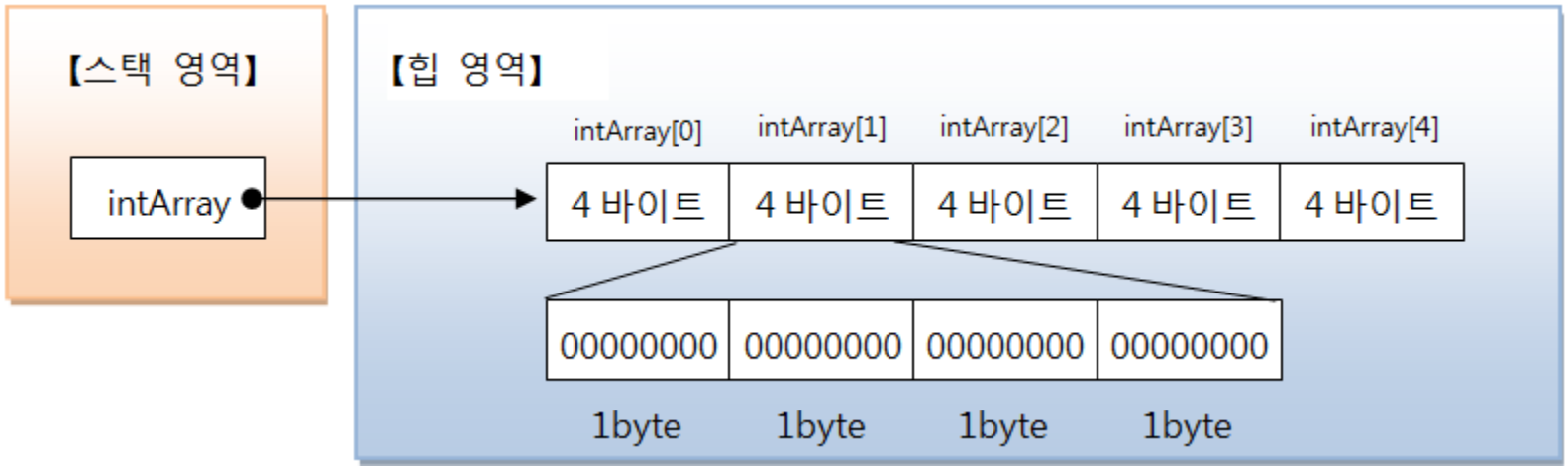
배열 생성 (2) – new 연산자

- new 연산자로 배열 생성
  - 향후 값들을 저장할 배열을 미리 생성
  - 디폴트 값으로 초기화된 상태로 생성됨

```
타입[] 변수 = new 타입[길이];
```

```
타입[] 변수 = null;
변수 = new 타입[길이];
```

```
int[] intArray = new int[5];
```



배열 생성 (3) – 타입별 기본값

- 타입 별 항목의 기본값

분류	데이터 타입	초기값
기본 타입 (정수)	byte[]	0
	char[]	'₩u0000'
	short[]	0
	int[]	0
	long[]	0L
기본 타입 (실수)	float[]	0.0F
	double[]	0.0
기본 타입 (논리)	boolean[]	false
참조 타입	클래스[]	null
	인터페이스[]	null

예제 3-1. 배열 선언 및 생성

양수 5개를 입력 받아 배열에 저장하고, 제일 큰 수를 출력하라

```
import java.util.Scanner;
public class ArrayAccess {
   public static void main(String[] args) {
      Scanner scanner = new Scanner(System.in);

      int[] intArray;
      intArray = new int[5];
      int max=0; // 현재 가장 큰 수
      System.out.println("양수 5개를 입력하세요.");

      for(int i=0; i<5; i++) {
         intArray[i] = scanner.nextInt();     // 입력 받은 정수를 배열에 저장
         if(intArray[i] > max)
            max = intArray[i];    // max 변경
      }
      System.out.print("가장 큰 수는 " + max + "입니다.");

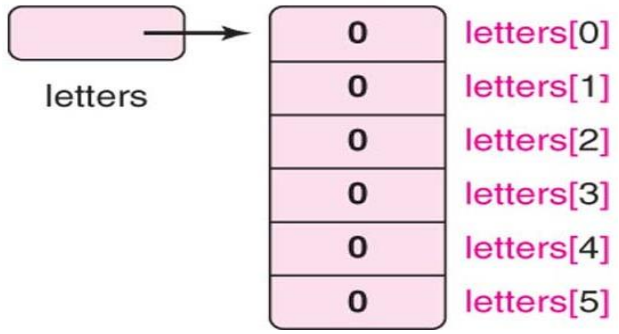
      scanner.close();
   }
}
```

```
양수 5개를 입력하세요.
1 39 78 100 99
가장 큰 수는 100입니다.
```

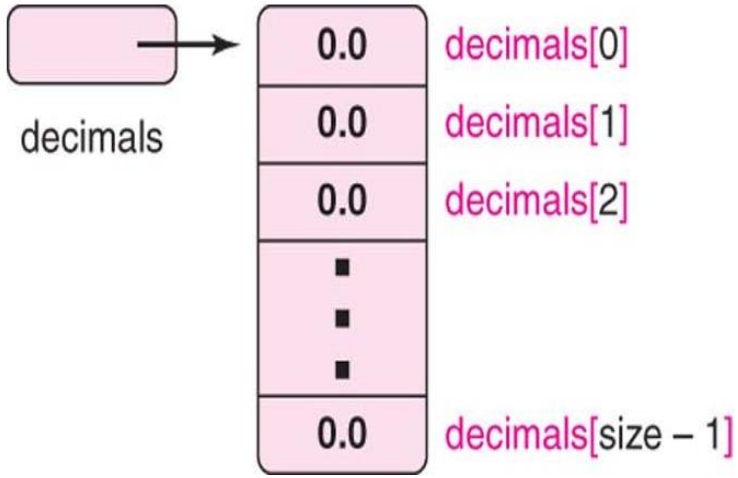
배열 생성 (4) – 크기의 동적 결정

- 프로그램 실행중에 배열 크기를 동적으로 결정할 수 있음
  - **단, 생성된 후 크기 변경 불가**

```
int x = 3;
char [] letters = new char[2*x];
```




```
System.out.println("Enter array size:  ");
int size = scanner.nextInt();
double [] decimals;
decimals = new double[size];
```

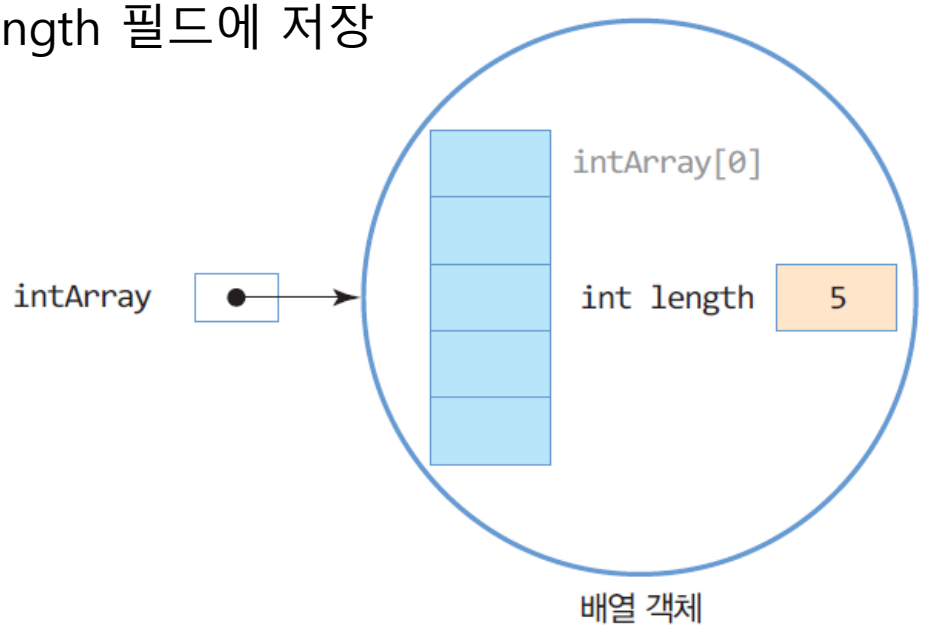


배열의 크기, length 필드

- 자바의 배열은 객체로 처리
  - 배열 객체의 length 필드
    - 배열의 크기는 배열 객체의 length 필드에 저장

```
int intArray[];
intArray = new int[5];

int size = intArray.length;
// size는 5
```



- length 필드를 이용하여 배열의 모든 값을 출력하는 사례

```
for(int i=0; i<intArray.length; i++) // intArray 배열 크기만큼 루프를 돈다.
    System.out.println(intArray[i]);
```

예제 3-2. 배열의 length 필드 활용

배열의 length 필드를 이용하여 배열 크기만큼 정수를 입력 받고 평균을 출력하라.

```
import java.util.Scanner;
public class ArrayLength {
   public static void main(String[] args) {
      Scanner scanner = new Scanner(System.in);

      System.out.println("5개의 정수를 입력하세요.");
      int[] intArray = new int[5];

      double sum=0.0;
      for(int i=0; i<intArray.length; i++)
         intArray[i] = scanner.nextInt(); // 키보드에서 입력받은 정수 저장

      for(int i=0; i<intArray.length; i++)
         sum += intArray[i]; // 배열에 저장된 정수 값을 더하기

      System.out.print("평균은 " + sum/intArray.length);
      scanner.close();
   }
}
```

```
5개의 정수를 입력하세요.
2 3 4 5 9
평균은 4.6
```

배열 사용 시 발생하는 예외

- NullPointerException
  - 배열 변수가 null인 상태에서 항목이나 length에 접근할 때 발생
  - 배열 객체가 아직 만들어지지 않았다는 의미
- ArrayIndexOutOfBoundsException
  - 인덱스가 0 ~ (length-1) 범위를 벗어날 때 발생
  - for 문 조건을 i <= length 로 쓰는 실수가 대표적

① 배열 변수가 null일 때

```
int[] intArray = null;
System.out.println(intArray[0]);
```

```
Exception in thread "main"
java.lang.NullPointerException
```

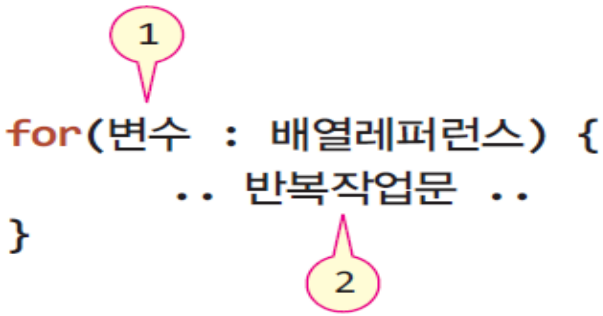
② 인덱스가 범위를 벗어날 때

```
int[] intArray = new int[5];
intArray[5] = 10;   // 유효 인덱스 0~4
```

```
Exception in thread "main"
java.lang.ArrayIndexOutOfBoundsException:
  Index 5 out of bounds for length 5
```

배열과 for-each 문

- for-each 문
  - 배열이나 나열(enumeration)의 원소를 순차 접근하는데 유용한 for 문



[주의] for-each 문의 제약
- 원소 값을 바꿔도 원본 배열에는 반영되지 않음 (기본 타입)
- 인덱스나 역순 접근이 필요하면 일반 for 문을 사용

  - for-each 문으로 정수 배열의 합을 구하는 사례

```
int [] n = { 1,2,3,4,5 };
int sum = 0;
for (int k : n) {
   sum += k;
}
```

반복될 때마다 k는 n[0], n[1], ..., n[4]로 번갈아 설정

for 문으로 구성하면 다음과 같다.

```
for(int i=0; i<n.length; i++) {
    int k = n[i];
    sum += k;
}
```

예제 3-3. for-each 문 활용

String [] **배열의 문자열을 출력하세요.**

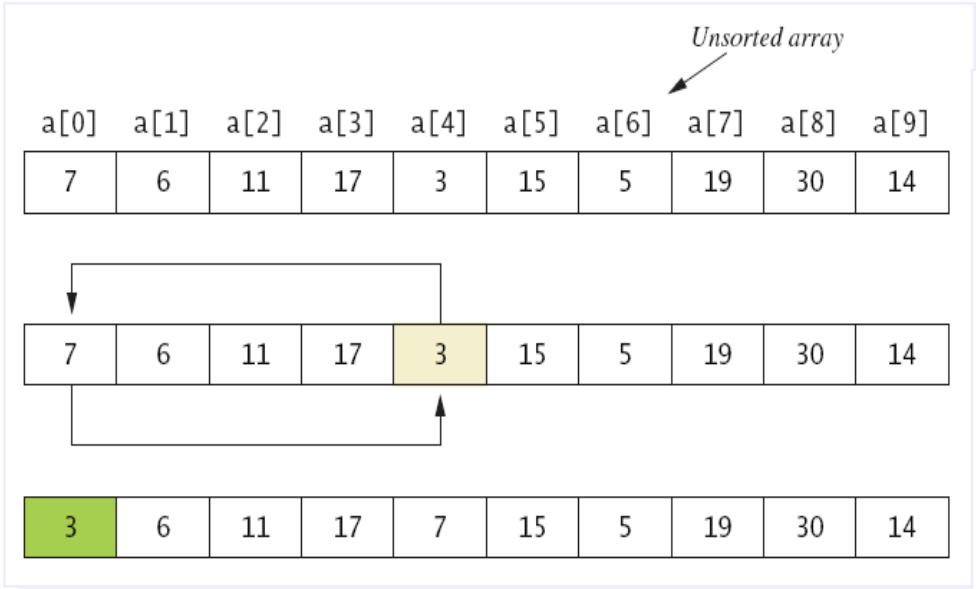
```
public class ForEachEx {
   public static void main(String[] args) {

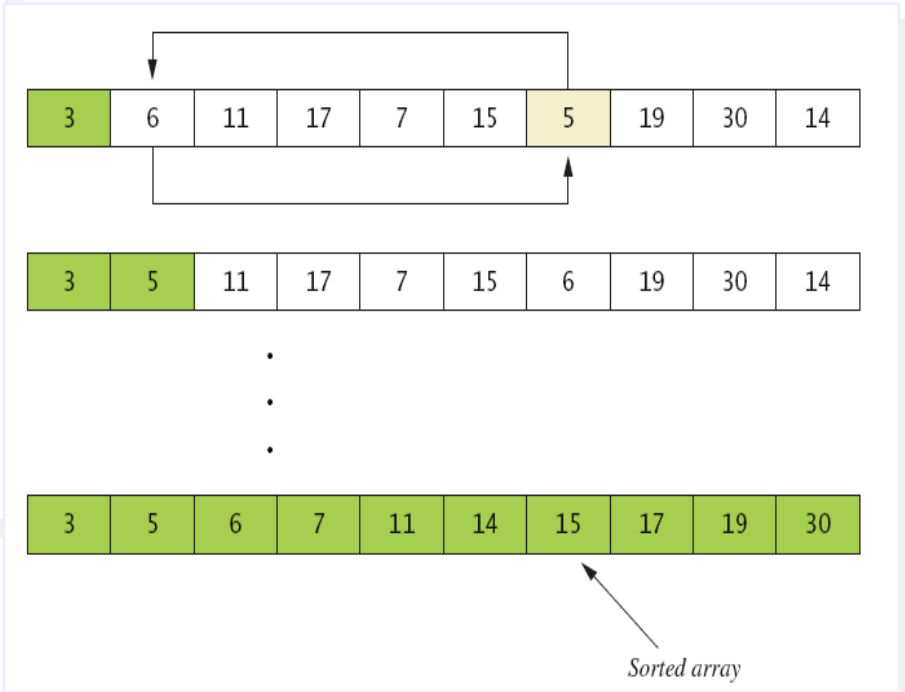
      String[] f = { "사과", "배", "바나나", "체리", "딸기", "포도" } ;
      for(String s : f) // s는 f[0], f[1], ..., f[5]로 반복
         System.out.print(s + " ");
   }
}
```

사과 배 바나나 체리 딸기 포도

실습 예제. 정렬하기

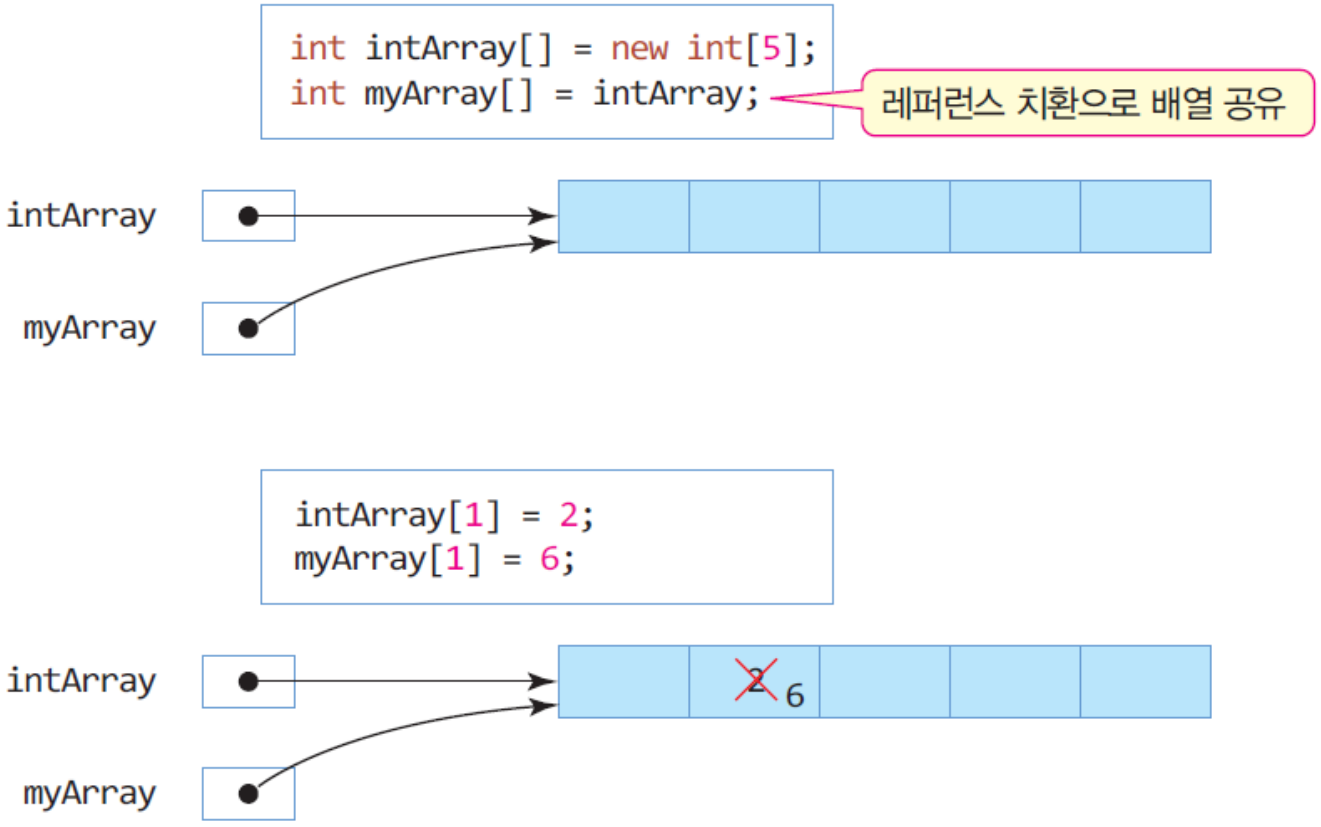
- 선택정렬 (Selection Sort)
  - 주어진 리스트 중에서 최소값의 위치를 찾는다
  - 그 값을 맨 앞에 위치한 값과 최소값을 교체한다
  - 맨 처음 위치를 뺀 나머지 리스트를 같은 방법으로 반복한다





배열 공유

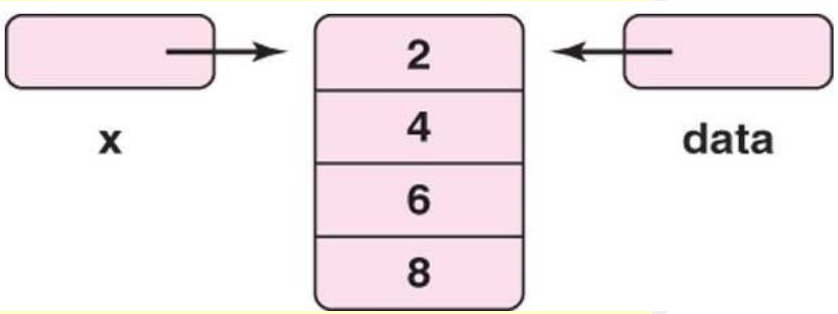
- 두 레퍼런스가 하나의 배열 공유



배열과 메소드

- 배열은 메소드의 매개변수로 전달될 수 있음

```
public static void changeData(int[] data)
{
     data[0]  = 200;
     data[1] = 400;
}
public static void main(String [] args)
{
     int[] x = {2, 4, 6, 8};
     changeData(x);
     System.out.print("x : "+ x[0] + ", " + x[1] + ", " +  x[2] + ", " + x[3]);
 }
```

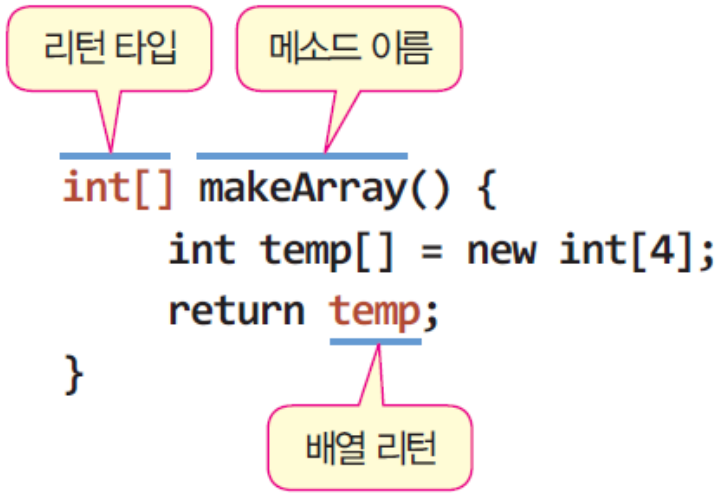


결과 ➔ x : 200, 400, 6, 8

메소드의 배열 리턴

- 배열 리턴
  - 배열의 레퍼런스만 리턴(배열 전체가 리턴되는 것이 아님)

- 메소드의 리턴 타입
  - 리턴하는 배열 타입과 리턴 받는 배열 타입 일치
  - **리턴 타입에 배열의 크기를 지정하지 않음**

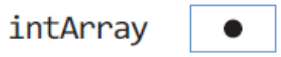


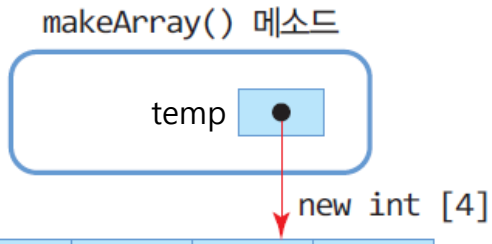

```
int[] intArray = makeArray();
```

배열을 리턴 받아 사용하는 과정

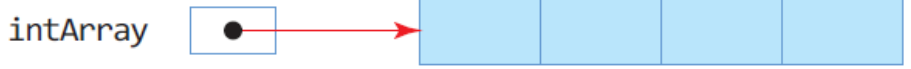
```
int[] makeArray() {
    int temp[] = new int[4];
    return temp;
}
```

(1) int[] intArray;

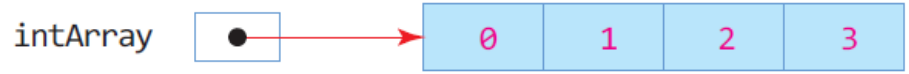




(2) intArray = makeArray();



(3) for(int i=0; i<intArray.length; i++)
intArray[i] = i;



배열 복사

- 배열 복사
  - 지정된 범위의 값들을 한 번에 통째로 복사
  - System.arraycopy(src, srcPos, dest, destPos, length)
    - src[srcPos]에서 dest[destPos]으로 length개의 데이터 복사
- 배열 복제
  - Object.clone() 메소드는 배열의 복제본을 생성해서 반환
    - clone()과 arraycopy() 모두 얕은 복사 – 객체 배열은 참조만 복사됨

```
int[] intArray = {1,2,3,4,5};
int[] copyArray = new int[10];
System.arraycopy(intArray, 0, copyArray, 0, intArray.length);
copyArray[0] = 10;
for(int num : intArray) {
  System.out.print(num+" ");
}
System.out.println();
for(int num : copyArray) {
  System.out.print(num+" ");
}
```

```
1 2 3 4 5
10 2 3 4 5 0 0 0 0 0
```

```
int[] cloneArray = intArray.clone();
cloneArray[0] = 100;

for(int num : cloneArray) {
  System.out.print(num+" ");
}
```

```
100 2 3 4 5
```

실습 예제. 로또 번호 생성기 만들기

- 1부터 45 사이의 로또 번호 6개를 생성하여 출력한다.
  - 단, 로또 번호는 중복되어서는 안된다.

<출력 예시>

로또 번호 : 12 8 28 41 36 40

난수 생성과 중복 검사

- 난수(random number) 생성 방법
  - Math.random() : 0.0 이상 1.0 미만의 double 반환 (import 불필요)
  - Random 클래스의 **nextInt(n)** : 0 이상 n 미만의 int 반환

Math.random() 사용

```
// 0.0 이상 1.0 미만의 실수
double d = Math.random();

// 1 이상 45 이하의 정수
int n = (int)(Math.random() * 45) + 1;
```

Random 클래스 사용

```
import java.util.Random;

Random rand = new Random();

// 0 이상 45 미만이므로 +1
int n = rand.nextInt(45) + 1;
```

수고하셨습니다.